

عنوان پروژه:

Mini Version Control System یا شبیه ساز گیت

طراحی و پیاده سازی یک سیستم کنترل نسخه ساده (Mini Git) با پایتون

- پروژه ی کارشناسی 1405 علوم کامپیوتر

- **دانشگاه :** دانشگاه سمنان

- **نام و نام خانوادگی دانشجو :** محمد مهدی میرزایی

- **شماره دانشجویی :** 40111360036

- **نام استاد راهنما :** سرکار خانم مارال صالحی

❖ پروژه توسط زبان برنامه نویسی پایتون (Python) طراحی شده است .

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

چکیده:

سیستم‌های کنترل نسخه یکی از ابزارهای مهم در فرآیند توسعه نرم‌افزار هستند که امکان ثبت، مدیریت و بازیابی نسخه‌های مختلف یک پروژه را فراهم می‌کنند. در این پروژه، یک سیستم کنترل نسخه ساده و آموزشی با عنوان Mini Git با استفاده از زبان برنامه‌نویسی Python طراحی و پیاده‌سازی شده است.

هدف اصلی پروژه، آشنایی عملی با مفاهیم اساسی سیستم‌های کنترل نسخه و پیاده‌سازی ساده برخی از قابلیت‌های اصلی Git بوده است. در این سیستم، قابلیت‌هایی مانند ایجاد Repository، محاسبه Hash برای فایل‌ها، ذخیره Object ها، ایجاد Staging Area، اضافه کردن فایل‌ها، ثبت Commit، مشاهده تاریخچه تغییرات، ایجاد Branch و بازیابی نسخه‌های قبلی پیاده‌سازی شده‌اند. ساختار پروژه به صورت ماژولار طراحی شده و هر بخش از برنامه در یک فایل مجزا قرار گرفته است. برای پیاده‌سازی پروژه از امکانات داخلی زبان Python از جمله کتابخانه‌های os، json، hashlib، datetime، shutil و argparse استفاده شده است. اطلاعات مربوط به Repository، Commit ها، Branch ها و فایل‌های Stage شده نیز به صورت محلی در ساختار mygit ذخیره می‌شوند.

نتیجه این پروژه، ایجاد یک نمونه ساده از یک سیستم کنترل نسخه خط فرمان است که اگرچه از نظر امکانات با Git واقعی قابل مقایسه نیست، اما مفاهیم اصلی و نحوه ارتباط بخش‌های مختلف یک سیستم کنترل نسخه را به صورت عملی نشان می‌دهد. انجام این پروژه همچنین موجب آشنایی بیشتر با مدیریت فایل‌ها، کار با JSON، Hashing، طراحی ماژولار و دریافت ورودی از خط فرمان در Python شده است.

فهرست مطالب :

چکید	3
فهرست مطالب	4
1. مقدمه	6
2. کارهای انجام شده	7
2.1 معرفی پروژه‌های مشابه	7
2.2 بررسی نمونه اول	7
2.3 بررسی نمونه دوم	8
2.4 بررسی نمونه سوم	9
2.5 مقایسه نمونه‌های بررسی شده	10
3. معرفی و ساختار کلی پروژه	12
3.1 ساختار Repository	14
3.2 توضیحات کدهای پروژه	16
3.2.1 ایجاد Repository	
3.2.2 پیاده‌سازی Hashing	
3.2.3 پیاده‌سازی Staging و Add	
3.2.4 پیاده‌سازی Commit	
3.2.5 پیاده‌سازی Log	
3.2.6 پیاده‌سازی Branch	
3.2.7 پیاده‌سازی Checkout	
3.2.8 نحوه اجرای پروژه	

3.2.9 تست و بررسی عملکرد پروژه

- 3.3 ارتباط بخش های مختلف پروژه 25
- 3.4 دستورات نهایی پروژه 27
- 3.5 توضیحات و جزئیات پروژه 28
4. جمع بندی و نتیجه گیری 31
5. محدودیت ها و قابلیت های قابل توسعه 32
6. منابع 33

1. مقدمه

در پروژه‌های برنامه‌نویسی، معمولاً فایل‌ها و کدهای یک نرم‌افزار در طول زمان بارها تغییر می‌کنند. در چنین شرایطی نگهداری نسخه‌های مختلف فایل‌ها، بررسی تغییرات و بازگشت به نسخه‌های قبلی اهمیت زیادی پیدا می‌کند. برای مدیریت این مسئله از سیستم‌های کنترل نسخه Version Control Systems استفاده می‌شود.

یکی از شناخته‌شده‌ترین سیستم‌های کنترل نسخه، Git است. Git امکان ذخیره نسخه‌های مختلف پروژه، ثبت تغییرات، ایجاد شاخه‌های مختلف و بازگشت به نسخه‌های قبلی را فراهم می‌کند. هدف این پروژه، طراحی و پیاده‌سازی یک نمونه ساده و آموزشی از سیستم کنترل نسخه با استفاده از زبان برنامه‌نویسی Python است. در این پروژه تلاش شده است برخی از مفاهیم اصلی Git به صورت ساده و قابل درک پیاده‌سازی شوند.

در پروژه حاضر، یک ابزار خط فرمان با نام Mini Git ایجاد شده است که کاربر می‌تواند از طریق Terminal با آن کار کند. این برنامه با استفاده از ساختار فایل‌ها و پوشه‌ها، اطلاعات مربوط به نسخه‌ها، فایل‌ها، Commitها و Branchها را مدیریت می‌کند.

هدف اصلی پروژه صرفاً ساخت یک جایگزین کامل برای Git نیست، بلکه آشنایی عملی با مفاهیمی مانند Hashing، Repository، Staging Area، Commit، Branch و Checkout و همچنین نحوه پیاده‌سازی آن‌ها با Python است.

2. کارهای انجام شده

معرفی پروژه‌های مشابه

پیش از طراحی و پیاده‌سازی پروژه، نمونه‌هایی از سیستم‌ها و پروژه‌های مرتبط با کنترل نسخه بررسی شدند تا نحوه عملکرد کلی این سیستم‌ها، ساختار Repository و ارتباط مفاهیمی مانند Commit، Branch، Object و Index بهتر شناخته شود.

مهم‌ترین نمونه‌های بررسی شده عبارت‌اند از:

- Git
- GitPython
- Dulwic

هدف از بررسی این نمونه‌ها، استفاده مستقیم از کد آن‌ها نبوده است؛ بلکه ساختار و مفاهیم اصلی آن‌ها به عنوان مبنایی برای طراحی یک نمونه ساده و آموزشی در پروژه حاضر مورد بررسی قرار گرفته‌اند.

نمونه اول Git:

Git یکی از شناخته‌شده‌ترین سیستم‌های کنترل نسخه است و برای مدیریت تغییرات پروژه‌های نرم‌افزاری مورد استفاده قرار می‌گیرد. Git یک سیستم کنترل نسخه توزیع‌شده است و مجموعه گسترده‌ای از دستورات را برای مدیریت Repository، ثبت تغییرات، ایجاد Branch، مشاهده تاریخچه و سایر عملیات مرتبط ارائه می‌کند.

از مهم‌ترین دستورات Git می‌توان به موارد زیر اشاره کرد:

git init

git add

git commit

git log

git branch

git checkout

این دستورات به ترتیب برای ایجاد Repository ، اضافه کردن تغییرات به Index ، ثبت Commit ، مشاهده تاریخچه ، مدیریت Branch ها و جابه جایی بین نسخه ها یا Branch ها استفاده می شوند. مستندات رسمی Git نیز این دستورات را در گروه های ایجاد Repository ، ثبت Snapshot ، Branching و مشاهده تاریخچه دسته بندی کرده است.

یکی از ویژگی های مهم Git ، استفاده از Object ها برای ذخیره اطلاعات Repository است. در ساختار Git ، Object های مختلفی مانند Blob ، Tree ، Commit و Tag وجود دارند و Object ها با استفاده از SHA-1 شناسایی می شوند.

بررسی Git در این پروژه اهمیت زیادی داشت؛ زیرا پروژه حاضر با هدف پیاده سازی یک نمونه ساده از برخی مفاهیم اصلی Git طراحی شده است.

نمونه دوم : GitPython

GitPython یک کتابخانه Python است که امکان کار با Repository های Git را از طریق Python فراهم می کند. این کتابخانه یک مدل شیء گرا برای دسترسی به اطلاعات Repository ارائه می دهد و امکاناتی مانند ایجاد Repository ، دسترسی به Branch ها ، Commit ها ، Index و Object های Git را فراهم می کند.

در GitPython می توان یک Repository را با استفاده از کلاس Repo مدیریت کرد:

```
from git import Repo
```

```
repo = Repo(path)
```

این کتابخانه امکانات مختلفی برای کار با Repository دارد. برای نمونه، می‌توان Repository جدید ایجاد کرد:

`Repo.init(path)`

همچنین GitPython امکان دسترسی به Branch فعال و اطلاعات مربوط به Commit ها را فراهم می‌کند. در این کتابخانه، HEAD به Branch فعال اشاره می‌کند و Branch ها نیز به Commit های مشخصی ارجاع می‌دهند.

یکی دیگر از بخش‌های مهم GitPython، Index است. Index در Git نقش Staging Area را دارد و تغییراتی را که قرار است در Commit بعدی قرار بگیرند نگهداری می‌کند. GitPython. نیز امکان اضافه کردن فایل‌ها به Index و ایجاد Commit از اطلاعات موجود در آن را فراهم می‌کند.

این پروژه از نظر ارتباط با Python اهمیت زیادی دارد، زیرا نشان می‌دهد که می‌توان مفاهیم Git را از طریق یک برنامه Python مدیریت کرد.

نمونه سوم: Dulwich

Dulwich یک پروژه Python مرتبط با Git است که با هدف ارائه یک رابط برای کار با Repository های Git ایجاد شده است. یکی از ویژگی‌های مهم Dulwich این است که می‌تواند به صورت مستقل و بدون نیاز مستقیم به نصب Git مورد استفاده قرار گیرد و بخش‌های مختلف Git را با Python پیاده‌سازی می‌کند.

Dulwich امکان کار با Repository های محلی و Remote را فراهم می‌کند و به عنوان یک پیاده‌سازی Python از بخش‌های مختلف Git شناخته می‌شود.

تفاوت مهم آن با GitPython این است که Dulwich به صورت مستقل از خود نرم‌افزار Git طراحی شده است، در حالی که GitPython علاوه بر امکانات Python، امکان اجرای مستقیم دستورات Git را نیز فراهم می‌کند.

بررسی Dulwich نشان داد که پیاده‌سازی مفاهیم اصلی Git با Python امکان‌پذیر است و می‌توان بدون ساخت یک نرم‌افزار کامل و پیچیده، بخش‌هایی از سازوکار داخلی سیستم‌های کنترل نسخه را به صورت آموزشی پیاده‌سازی کرد.

مقایسه نمونه‌های بررسی شده

نمونه‌های بررسی شده از نظر هدف و سطح پیچیدگی با یکدیگر تفاوت دارند.

ویژگی اصلی	زبان/فناوری	نوع پروژه	نمونه
سیستم کامل و حرفه‌ای کنترل نسخه	C و سایر اجزا	سیستم کنترل نسخه واقعی	Git
دسترسی برنامه‌نویسی به Repository های Git	Python	کتابخانه	GitPython
پیاده‌سازی مستقل Git با Python	Python	پیاده‌سازی Git	Dulwich
پیاده‌سازی ساده و آموزشی مفاهیم اصلی	Python	پروژه حاضر	Mini Git

Git امکانات بسیار گسترده‌ای برای استفاده حرفه‌ای در پروژه‌های نرم‌افزاری دارد و شامل قابلیت‌هایی مانند Branch، Merge، Remote، Tag، Diff و مدیریت تاریخچه است.

GitPython بیشتر نقش یک واسط برنامه‌نویسی برای کار با Git را دارد و Dulwich تلاش می‌کند بخش‌های Git را با Python و به صورت مستقل پیاده‌سازی کند.

در مقایسه با این نمونه‌ها، هدف پروژه حاضر ایجاد یک سیستم کامل و جایگزین Git نیست؛ بلکه هدف، پیاده‌سازی بخش‌هایی از مفاهیم اصلی سیستم کنترل نسخه در قالب یک پروژه ساده Python است. به همین دلیل، قابلیت‌های پروژه محدودتر از Git و کتابخانه‌های حرفه‌ای مرتبط با آن در نظر گرفته شده‌اند.

نتیجه بررسی نمونه‌های مشابه

بررسی نمونه‌های موجود نشان داد که مفاهیمی مانند Repository، Object، Hash، Index یا Branch، Commit، Staging Area و HEAD از اجزای مهم معماری سیستم‌های کنترل نسخه هستند. در Git، Objectها با شناسه‌های مبتنی بر SHA-1 شناسایی می‌شوند و Index نقش Stage مربوط به تغییرات را دارد. همچنین Branchها و HEAD برای مشخص کردن مسیر و موقعیت فعلی پروژه استفاده می‌شوند.

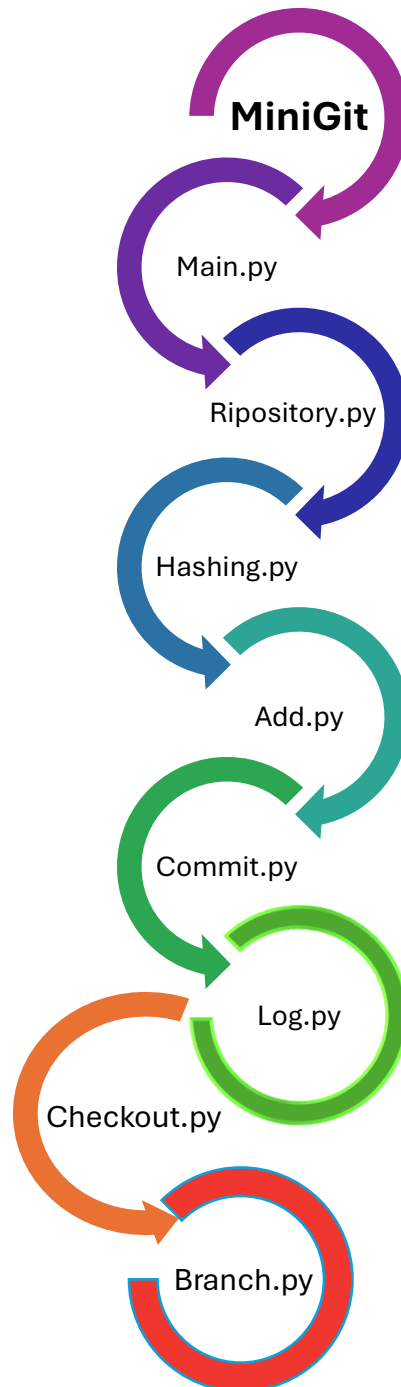
این بررسی به عنوان مرحله مطالعاتی پروژه انجام شد و مبنایی برای انتخاب قابلیت‌هایی قرار گرفت که در نسخه آموزشی پروژه Mini Git پیاده‌سازی شدند.

در بخش بعد، ساختار و نحوه پیاده‌سازی پروژه شخصی با عنوان Mini Git و اجزای مختلف آن توضیح داده خواهد شد.

3. معرفی و ساختار کلی پروژه

پروژه به صورت یک Command Line Application طراحی شده است. بنابراین برای استفاده از آن نیازی به رابط گرافیکی وجود ندارد و دستورات از طریق Terminal وارد می‌شوند.

ساختار اصلی پروژه به صورت زیر است :



هر فایل مسئولیت مشخصی دارد و بخش‌های مختلف پروژه در فایل‌های جداگانه قرار گرفته‌اند.

main.py

فایل اصلی برنامه است و وظیفه دریافت دستورات کاربر از Terminal و اجرای تابع مربوط به هر دستور را بر عهده دارد.

Repository.py

برای ایجاد Repository و ساخت ساختار اولیه پوشه mygit. استفاده می‌شود.

hashing.py

وظیفه محاسبه Hash فایل‌ها و ذخیره‌Object های مربوط به آن‌ها را بر عهده دارد.

add.py

برای اضافه کردن فایل‌ها به Staging Area استفاده می‌شود.

commit.py

نسخه فعلی فایل‌های Stage شده را در قالب یک Commit ذخیره می‌کند.

log.py

تاریخچه Commit های پروژه را نمایش می‌دهد.

checkout.py

برای بازگرداندن فایل‌ها به وضعیت یک Commit مشخص استفاده می‌شود و در نسخه توسعه‌یافته پروژه برای جابه‌جایی بین Branch ها نیز در نظر گرفته شده است.

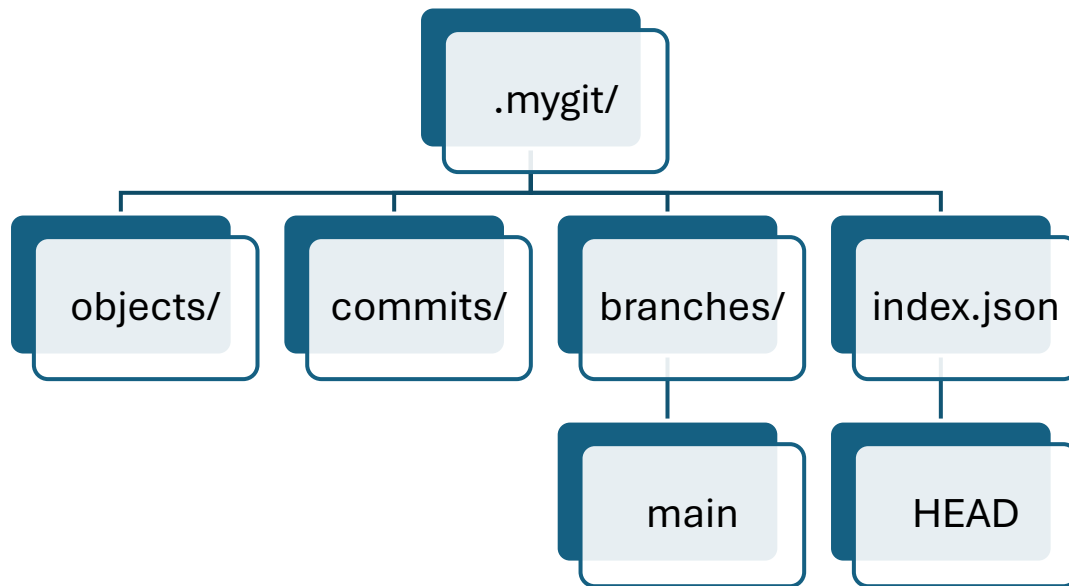
branch.py

وظیفه ایجاد و نمایش شاخه های پروژه را بر عهده دارد.

ساختار Repository

پس از اجرای دستور `init` ، برنامه پوشه‌ای به نام `mygit` ایجاد می‌کند.

ساختار آن به شکل زیر است:



هر بخش وظیفه خاصی دارد.

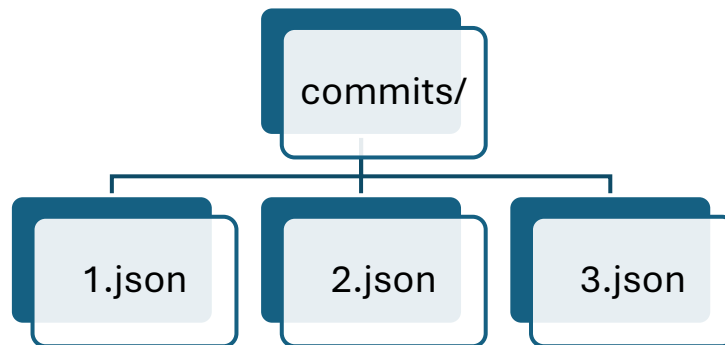
objects

در این پوشه نسخه ذخیره‌شده فایل‌ها بر اساس Hash آن‌ها قرار می‌گیرد.

commits

اطلاعات مربوط به Commit ها در این پوشه ذخیره می‌شوند.

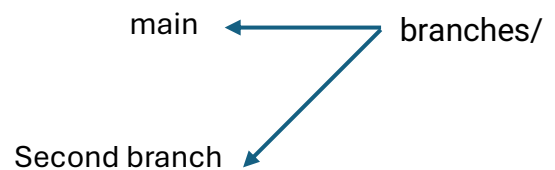
برای مثال:



branches

اطلاعات مربوط به Branch ها در این پوشه نگهداری می شود.

برای مثال:



index.json

این فایل نقش Staging Area پروژه را دارد و اطلاعات فایل هایی که با دستور add انتخاب شده اند در آن ذخیره می شود.

HEAD

این فایل مشخص می کند که در حال حاضر کدام Branch فعال است.

برای مثال:

main

توضیحات کدهای پروژه

ایجاد Repository

اولین بخش پروژه، ایجاد یک Repository محلی بود.

برای این کار تابع `init_repository()` نوشته شد:

```
definit_repository():
```

```
os.makedirs(".mygit",exist_ok=True)
```

```
os.makedirs(".mygit/objects",exist_ok=True)
```

```
os.makedirs(".mygit/commits",exist_ok=True)
```

```
os.makedirs(".mygit/branches",exist_ok=True)
```

در این قسمت پوشه اصلی `.mygit` و سه پوشه `objects`، `commits` و `branches` ساخته می‌شوند.

همچنین فایل‌های `index.json` و `HEAD` ایجاد شده و Branch اولیه با نام `main` ساخته می‌شود.

کاربر برای ایجاد Repository می‌تواند دستور زیر را اجرا کند:

```
py main.py init
```

Hash کردن فایل‌ها

یکی از بخش‌های مهم پروژه، تولید Hash برای فایل‌ها است.

برای این کار از کتابخانه داخلی Python به نام hashlib استفاده شده است.

```
import hashlib
```

فایل ابتدا به صورت Binary خوانده می‌شود:

```
with open(filepath, "rb") as f:
```

```
    content=f.read()
```

سپس Hash مربوط به محتوای فایل ایجاد می‌شود:

```
file_hash=hashlib.sha1(content).hexdigest()
```

در این پروژه از الگوریتم SHA-1 برای تولید Hash استفاده شده است.

مزیت این روش در پروژه حاضر این است که می‌توان یک فایل را با یک شناسه مشخص شناسایی کرد.

اگر محتوای دو فایل کاملاً یکسان باشد، Hash آن‌ها نیز یکسان خواهد بود.

ذخیره Object ها

پس از تولید Hash، فایل در پوشه objects ذخیره می‌شود.

مسیر Object به صورت زیر ساخته می‌شود:

```
object_path=os.path.join(
```

```
    ".mygit",
```

```
    "objects",
```

```
    file_hash)
```

بنابراین برای مثال اگر Hash فایل:

abc123...

باشد، Object مربوط به آن در مسیری مانند زیر قرار می‌گیرد:

.mygit/objects/abc123...

همچنین قبل از ذخیره فایل بررسی می‌شود که Object مربوط به آن قبلاً وجود نداشته باشد:

```
if not os.path.exists(object_path):
```

این کار باعث می‌شود در صورت یکسان بودن محتوای فایل‌ها، Object تکراری ایجاد نشود.

Add و Staging Area دستور

مرحله بعدی پروژه، پیاده‌سازی دستور add بود.

هدف Staging Area این است که فایل‌هایی را که قرار است در Commit بعدی قرار بگیرند مشخص کند.

در پروژه، index.json نقش Staging Area را دارد.

تابع اصلی این بخش:

```
def add_file(filepath):
```

ابتدا فایل به Object تبدیل شده و Hash آن دریافت می‌شود:

```
file_hash = save_object(filepath)
```

سپس فایل index.json خوانده می‌شود:

```
with open(".mygit/index.json", "r") as f:
```

```
    index = json.load(f)
```

و فایل به همراه Hash آن در Dictionary قرار می‌گیرد:

```
Index[filepath] = file_hash
```

در نهایت اطلاعات جدید دوباره در index.json ذخیره می‌شود.

کاربر می‌تواند با دستور زیر یک فایل را به Staging Area اضافه کند:

```
py main.py add test.txt
```

ایجاد Commit

پس از اضافه کردن فایل‌ها به Staging Area ، امکان ثبت یک نسخه از پروژه با استفاده از Commit فراهم می‌شود.

تابع:

```
create_commit(message)
```

اطلاعات موجود در index.json را دریافت می‌کند.

اگر هیچ فایلی در Staging Area نباشد:

```
If len (index)==0:
```

```
Print ("Nothing to commit")
```

```
return
```

پیغام مناسب نمایش داده می‌شود.

سپس شماره Commit تعیین می‌شود:

```
commit_files = os.listdir (".mygit/commits")
```

```
commit_id = len (commit_files) + 1
```

اطلاعات Commit در یک Dictionary ذخیره می‌شود:

```
commit_data = {  
    "id": commit_id,  
    "message": message,  
    "timestamp": str(datetime.now()),  
    "files": index  
}
```

در این اطلاعات موارد زیر ذخیره می‌شوند:

- شماره Commit
- پیام Commit
- زمان ایجاد Commit
- فایل‌های موجود در آن Commit
- Hash فایل‌ها

سپس اطلاعات در یک فایل JSON ذخیره می‌شود.

مثلاً:

.mygit/commits/1.json

دستور ایجاد: Commit

```
py main.py commit "First version"
```

مدیریت Branch

یکی دیگر از قابلیت‌های پیاده‌سازی شده، Branch است.

Branch امکان ایجاد مسیرهای جداگانه برای توسعه پروژه را فراهم می‌کند.

برای مثال:

main

second branch

در پروژه هر Branch به صورت یک فایل در پوشه زیر ذخیره می‌شود:

.mygit/branches/

برای ایجاد Branch جدید:

py main.py branch second branch

Branch جدید ایجاد می‌شود.

همچنین با اجرای:

py main.py branch

Branch های موجود نمایش داده می‌شوند.

Branch فعال با * مشخص می‌شود:

* main

second branch

فایل HEAD

فایل HEAD برای مشخص کردن Branch فعلی استفاده می‌شود.

برای مثال اگر محتویات آن:

main

باشد، Branch فعلی main است.

در هنگام Commit، برنامه ابتدا Branch موجود در HEAD را می‌خواند و سپس اطلاعات آخرین Commit را برای آن Branch ذخیره می‌کند.

این موضوع باعث می‌شود ارتباط زیر شکل بگیرد:

HEAD

↓

main

↓

Commit

در نتیجه Branch می‌تواند مشخص کند که آخرین نسخه ثبت‌شده آن کدام Commit است.

نمایش تاریخچه Commit ها با Log

برای مشاهده Commit های ثبت‌شده، قابلیت log ایجاد شد.

دستور:

```
py main.py log
```

تمام فایل‌های موجود در پوشه:

.mygit/commits

خوانده می‌شوند.

برای مرتب‌سازی Commit ها، تابعی به نام:

Def get_commit_number (filename):

نوشته شد.

این تابع پسوند json را از نام فایل حذف کرده و شماره Commit را به عدد تبدیل می‌کند:

number= filename.replace(".json", "")

return int (number)

سپس Commit ها از جدیدترین به قدیمی‌ترین مرتب می‌شوند.

نمونه خروجی:

Commit #2

Message : Second version

Date : 2026-08-09 12:30:20

Commit #1

Message : First version

Date : 2026-08-09 12:10:15

Checkout

یکی دیگر از قابلیت‌های مهم پروژه، checkout است. هدف این بخش، بازگرداندن فایل‌ها به وضعیتی است که در یک Commit مشخص ثبت شده‌اند. برای مثال:

```
py main.py checkout 1
```

برنامه فایل مربوط به Commit شماره 1 را پیدا می‌کند:

`.mygit/commits/1.json`

سپس Hash فایل‌های موجود در Commit را می‌خواند.

با استفاده از Hash، Object مربوط به هر فایل در مسیر:

`.mygit/objects/`

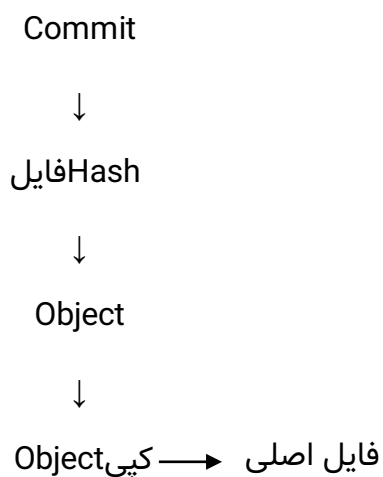
پیدا می‌شود.

در نهایت Object دوباره به فایل اصلی کپی می‌شود.

برای کپی کردن فایل‌ها از کتابخانه `shutil` و تابع زیر استفاده شده است:

`Shutil.copyfile(object_path,filepath)`

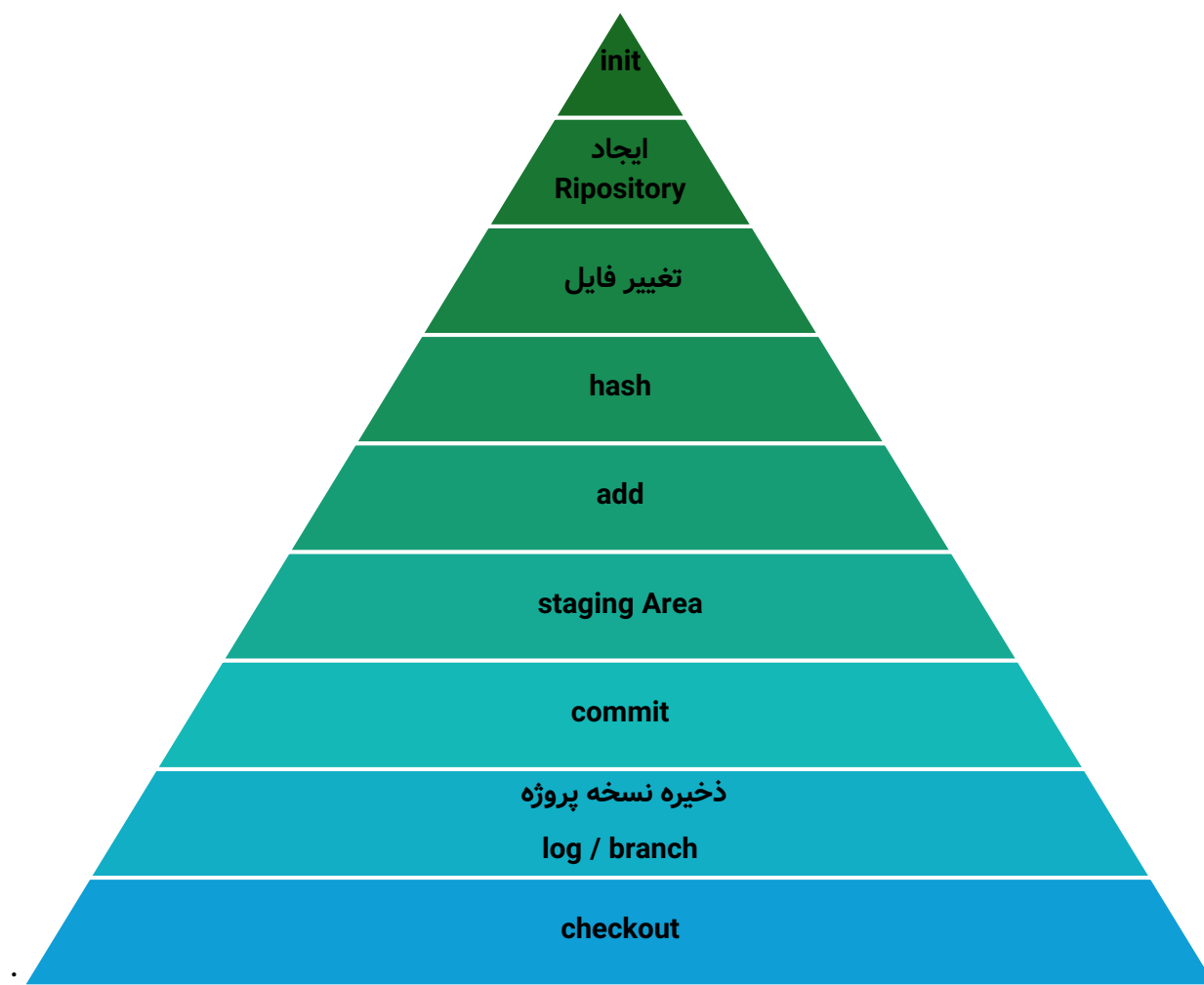
به صورت مفهومی:



به این ترتیب فایل‌ها به وضعیت ذخیره‌شده در Commit مورد نظر برمی‌گردند.

ارتباط بخش‌های مختلف پروژه

روند کلی استفاده از Mini Git به صورت زیر است:



استفاده از argparse

برای دریافت دستورات کاربر از Terminal از کتابخانه داخلی argparse استفاده شد.

```
import argparse
```

Parser ایجاد شده و دو آرگومان برای برنامه تعریف شد:

```
parser = argparse.ArgumentParser()
```

```
parser.add_argument("command")
```

```
parser.add_argument("arg", nargs="?")
```

آرگومان اول مشخص کننده دستور است و آرگومان دوم اختیاری است.

برای مثال:

```
py main.py add test.txt
```

در این حالت:

`args.command`

برابر با:

`add`

و

`args.arg`

برابر با:

`test.txt`

خواهد بود.

با این روش تمام دستورات از طریق یک فایل اصلی مدیریت می شوند.

دستورات نهایی پروژه

در نسخه فعلی، دستورات اصلی پروژه به صورت زیر هستند:

دستور	کاربرد
init	ایجاد Repository
Hash	تولید Hash و ذخیره Object
Add	اضافه کردن فایل به Staging Area
Commit	ثبت یک نسخه جدید
Log	نمایش تاریخچه Commit ها
Branch	ایجاد و نمایش Branch
Checkout	بازگرداندن وضعیت یک Commit

نمونه دستورات:

```
py main.py init
```

```
py main.py hash test.txt
```

```
py main.py add test.txt
```

```
py main.py commit "First version"
```

py main.py log

py main.py branch second branch

py main.py branch

py main.py checkout 1

توضیحات و جزئیات پروژه

در این پروژه بخش اصلی کار شامل طراحی و پیاده‌سازی یک سیستم کنترل نسخه ساده با استفاده از Python بود.

در طول انجام پروژه، ساختار برنامه به صورت چند فایل مستقل طراحی شد تا هر قسمت وظیفه مشخصی داشته باشد. این روش باعث شد کد پروژه قابل فهم‌تر و مدیریت آن ساده‌تر شود.

مهم‌ترین بخش‌های انجام‌شده در پروژه شامل موارد زیر است:

- ایجاد ساختار Repository
- ایجاد و مدیریت فایل‌ها و پوشه‌های داخلی mygit.
- پیاده‌سازی Hashing با استفاده از hashlib
- ذخیره‌Object های فایل
- پیاده‌سازی Staging Area
- پیاده‌سازی دستور add
- پیاده‌سازی سیستم Commit
- ذخیره اطلاعات Commit در JSON
- پیاده‌سازی تاریخچه با log
- ایجاد سیستم Branch

- پیاده‌سازی Checkout
 - اتصال تمامی قابلیت‌ها به یک برنامه خط فرمان
 - استفاده از argparse برای دریافت دستورات
- در کنار پیاده‌سازی قابلیت‌ها، مفاهیم مختلف زبان Python نیز در این پروژه مورد استفاده و تمرین قرار گرفتند.

از جمله:

- Function
- Dictionary
- List
- شرط‌ها و حلقه‌ها
- File Handling
- with open()
- JSON
- Exception‌های مربوط به فایل‌ها
- کار با مسیرها با os
- os.path
- os.listdir
- os.makedirs
- shutil
- datetime
- hashlib
- argparse

- کار با Module های مختلف Python

یکی از چالش‌های پروژه، مدیریت ارتباط بین فایل‌های مختلف و نحوه ذخیره و بازیابی اطلاعات بود. همچنین هنگام توسعه پروژه، خطاهای مربوط به مسیر فایل‌ها، پیدا نشدن فایل و نحوه دریافت آرگومان‌های خط فرمان بررسی و برطرف شدند.

4. جمع‌بندی و نتیجه‌گیری

در این پروژه یک سیستم کنترل نسخه ساده با نام Mini Git با استفاده از زبان برنامه‌نویسی Python طراحی و پیاده‌سازی شد.

هدف پروژه، آشنایی عملی با نحوه عملکرد سیستم‌های کنترل نسخه و پیاده‌سازی ساده مفاهیم اصلی Git بود.

در پایان پروژه، برنامه قادر است یک Repository ایجاد کند، فایل‌ها را با استفاده از Hash شناسایی و ذخیره کند، فایل‌ها را در Staging Area قرار دهد، نسخه‌های مختلف پروژه را به صورت Commit ذخیره کند، تاریخچه نسخه‌ها را نمایش دهد و امکان ایجاد Branch و بازگرداندن فایل‌ها به نسخه‌های قبلی را فراهم کند.

یکی از مهم‌ترین نتایج این پروژه، درک بهتر ارتباط میان مفاهیم مختلف سیستم کنترل نسخه است. برای مثال، فایل به وسیله Hash شناسایی می‌شود، Hash در Staging Area ثبت می‌شود، اطلاعات Commit در Staging ذخیره می‌شود و Branch نیز مشخص می‌کند که کدام Commit در مسیر توسعه فعلی قرار دارد.

البته این پروژه یک نسخه آموزشی و ساده‌شده از Git است و با Git واقعی تفاوت‌های زیادی دارد. برای مثال Git واقعی دارای الگوریتم‌ها و ساختارهای پیچیده‌تر، مدیریت حرفه‌ای تر Branch‌ها، Merge، Diff، Remote Repository، سیستم Object پیشرفته‌تر و قابلیت‌های بسیار بیشتری است. با این حال، پیاده‌سازی Mini Git باعث شد مفاهیم اصلی Version Control از حالت تئوری خارج شده و به صورت عملی در قالب یک پروژه Python پیاده‌سازی شوند.

5. محدودیت‌ها و قابلیت‌های قابل توسعه

نسخه فعلی پروژه با هدف آموزشی طراحی شده و قابلیت‌های آن محدود به امکانات اصلی است. برخی قابلیت‌هایی که در آینده می‌توان به پروژه اضافه کرد عبارت‌اند از:

- پیاده‌سازی diff
- نمایش دقیق تغییرات بین دو نسخه
- Merge کردن Branch ها
- حذف Branch
- مدیریت بهتر Commit ها
- بررسی تغییرات فایل‌ها به صورت دقیق‌تر
- پیاده‌سازی سیستم بهتر برای مدیریت نسخه‌ها
- اضافه کردن رابط گرافیکی
- اضافه کردن تست‌های خودکار
- بهبود مدیریت خطاها

این موارد می‌توانند در نسخه‌های آینده باعث نزدیک‌تر شدن Mini Git به عملکرد یک سیستم کنترل نسخه واقعی شوند.

6. منابع

۱. آموزش جامع Git به زبان فارسی — GitHub

توضیح:

این منبع یک آموزش فارسی برای یادگیری Git از مفاهیم پایه تا مباحث پیشرفته است. موضوعاتی مانند Working Directory ، Staging Area ، Commit ، Branch و Remote را پوشش می‌دهد و برای آشنایی با نحوه کار سیستم‌های کنترل نسخه و دستورات اصلی Git مورد استفاده قرار گرفته است.

لینک سایت:

<https://github.com/AsaEdgerunner/git-course-fa>

۲. کتاب جامع Git و GitHub — A-Talebifard

توضیح:

این کتاب یک مرجع فارسی برای یادگیری Git و GitHub است و مباحثی مانند مفاهیم پایه Git ، Repository ، Staging Area ، دستورات add ، init ، commit ، log ، diff و Branch و سایر مفاهیم مرتبط با سیستم‌های کنترل نسخه را بررسی می‌کند. این منبع برای مطالعه مفاهیم و دستورات مورد استفاده در پروژه Mini Git استفاده شده است.

لینک سایت:

<https://a-talebifard.github.io/git-github-book/>

۳. آموزش مبانی Git — فراگیری

توضیح:

این منبع یک آموزش فارسی و مرحله‌به‌مرحله برای آشنایی با Git است. مباحثی مانند مفهوم Git ، معماری Git ، ایجاد Repository ، اعمال تغییرات ، ثبت نسخه‌ها و مدیریت Branch ها را بررسی می‌کند و برای درک مفاهیم پایه سیستم‌های کنترل نسخه مورد استفاده قرار گرفته است.

لینک سایت:

<https://faragiri.com/learning/git/>

4. دوره ی آشنایی با Git مکتب خونه توسط جادی

5. استفاده از ابزار های هوش مصنوعی مانند Chatgpt و Deep seek